

1. Application Architecture

- **Original Application:** Must consist of a Frontend (make it nice), a Database (**AWS RDS only**), and a Backend with at least 3 microservices.
- **Frontend Requirements:** Must be accessible via a custom **DNS** name with **SSL** (HTTPS) fully configured.
- **Infrastructure as Code (IaC):** Every resource (EKS, RDS, VPC) must be managed **exclusively via Terraform**. Day 1 (initial setup) and Day 2 (updates) must be fully automated.

2. Deployment & CI/CD (Git-Driven Promotion)

- **Promotion Flow:** Dev --> Nightly Build (QA) --> UAT --> Prod.
- **Automation Logic:**
 - **Dev/QA to UAT:** Must be triggered automatically via **Conventional Commits** (e.g., **RC1**, **RC2**, ...) or Pull Request merges.
 - **UAT to Production:** Must be triggered via **Release Labels/Tags** (e.g., **v1.0.1**). Manual "click-to-deploy" in the AWS Console is prohibited.
- **Strategy:** Pick and justify either **Blue/Green** or **Canary** for EKS.
- **Zero Downtime:** All promotions and updates must result in zero dropped requests.

3. Mandatory "Day 2" Scenarios

Students must demonstrate these scenarios live or via narrated video. They will be graded on the **logic and thought process** used to maintain stability:

1. **OS/Security Patching:** Update the underlying EC2 worker nodes (AMIs) with the latest patches without interrupting service.
2. **Schema Changes:** Deploy a backend change that updates the RDS database schema.
 - **Requirement:** Students must explain and demonstrate how DB schema changes are applied.

4. Observability & Logging (Self-Hosted Only)

- **Monitoring:** Deploy **Prometheus** and **Grafana** inside the cluster (No AWS Managed services).
 - Dashboard must track CPU, Memory, and Disk Space for all nodes.
 - **Alerts:** Email (and/or Slack) notifications for critical resource thresholds.
 - **External Access & Security:** Grafana must be reachable from outside AWS. **Username/Password is prohibited**; you must implement **OAuth2** (Okta, GitHub, or Google) for dashboard access.
- **Centralized Logging:** Deploy your own stack (**Loki** or **ELK/OpenSearch**) or use a self-hosted 3rd party (e.g., **Sentry**) on EKS.
 - Must support centralized queries across the backend and all 3 microservices.